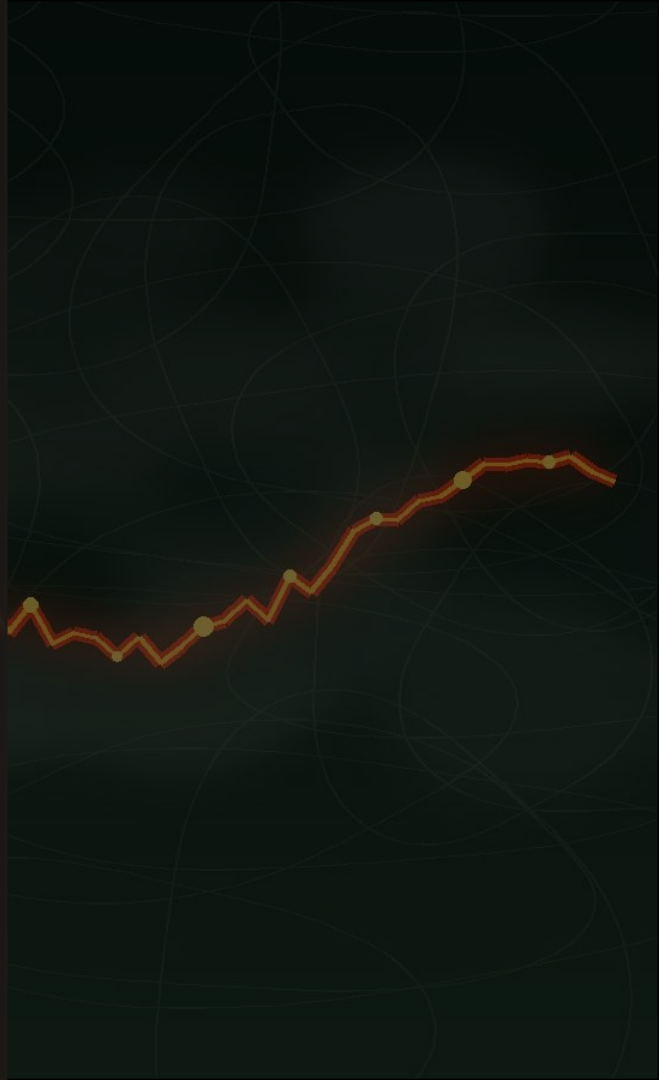
GHCP DEV DAY · COMMUNITY SHOWCASE

Ops Fire Fabric

Ontario wildfire intelligence: satellites, weather and official data fused on Microsoft Fabric — and deployed end to end by GitHub Copilot



Parth Vyas
Live demo on an F32 Fabric capacity in an MCAPS subscription

01 · WHY

Wildfire evidence is fragmented

- Every sensor sees fire differently — heat, burn scars, radar change — at different resolutions and times
- Smoke and cloud blind the optical satellites exactly when fires are worst
- Official perimeters are authoritative but lag behind the satellite passes
- Weather and fire-danger indices sit in yet another system
- Analysts end up stitching downloads, notebooks and GIS files by hand

4

satellite sensors, from 10 m to 1 km resolution

~6/day

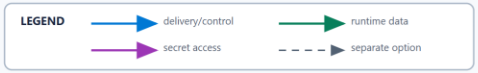
VIIRS active-fire passes over Ontario

1 map

the goal: one current, explainable view

Ontario Wildfire Intelligence

Azure DevOps to Microsoft Fabric solution architecture | repository evidence as of 2026-08-25



DELIVERY CONTROL PLANE

Validated source, independent approval, exact-SHA deployment, serialized schedule, and recoverable capacity lifecycle

Azure Repos
main-ado
/azure-pipelines.yml

main-ado

Build + validate
tests, manifests, F32 plan
immutable source artifact

artifact

Approve
manual validation +
Environment checks

approved

Deploy + optional accept
WIF definitions by default
exact-SHA run only when approved

federated

ADO WIF identity
workspace Admin
KV Secrets User

SCHEDULED: exact submit > agentless 60m wait > fresh-agent result gate + capacity recovery

Fabric REST APIs
definitions + approved run

DATA SOURCES

Public and official upstream systems

01 **Ontario MNRF**
active fires + official
perimeters + validation

02 **Planetary Computer**
Sentinel-1 SAR
Sentinel-2 L2A
Sentinel-3

03 **NASA FIRMS**
VIIRS: SNPP, NOAA-20,
NOAA-21

04 **Weather services**
Open-Meteo observations
+ CWFIS

MICROSOFT FABRIC DATA PLANE

Target workspace p-mcaps-bthan-opsfire on F32 capacity | OneLake-backed medallion architecture

Fabric workspace
item boundary + access
custom pools: Admin

Daily Data Pipeline
13 notebook activities
explicit dependencies

Environments + pools
geo-medium, geo-large,
geo-xlarge profiles

ops_fire_config
258 non-secret defaults
Key Vault URL + secret name

Fabric notebooks + Spark
19 deployed notebooks
ingest MNRF/weather
process S1/S2/VIIRS/S3 + FWI
fuse boundary + forecast spread
publish Gold + PMTiles

ONE PIPELINE RUN

Ingest	Process	Publish
AOI • perimeters • weather	satellites • FWI • boundary boundary + FWI → spread	Gold • tiles

OneLake / Delta Lake storage

Bronze
raw API payloads
and source snapshots

Silver
SAR, burn scars, VIIRS,
FRP, weather/FWI

Gold
boundary + spread
GeoJSON
release-ready layers

Historic
run history +
cumulative products

Gold layers

PUBLICATION + CONSUMPTION

Immutable release contract
viewer hosting deploys separately

Immutable PMTiles release
run_manifest.json pins one release
13 configured layers + optional JSON/GeoJSON

manifest + Range

React + Leaflet viewer
layer controls, filters, playback, inspection
HTTP byte-range requests

optional deploy

Azure Static Web Apps
separately deployable hosting target
not provisioned by this repository pipeline

Operations users
browser map consumers

IDENTITY + SECRET ACCESS

Vault-scoped least privilege; verify the Fabric pipeline last-modified identity after each ADO deployment

Azure Key Vault
FIRMS secret value
read at notebook runtime

get secret

Approved runtime identities
ADO WIF principal • workspace identity baseline
human/SP/managed identity only while executing
Key Vault Secrets User at vault scope

Separation of duties
Secrets Officer: time-bound rotation
Data Access Admin: time-bound RBAC
Build Service + approver: no Key Vault role

Release gate
audit inherited roles + groups
verify ADO as last-modifier candidate
reject stale or overbroad access

last-modifier candidate

An evidence-integration pipeline and a map

A daily Fabric data product

22 Spark notebooks, orchestrated by one Fabric Data Pipeline, ingest, process, fuse and publish Ontario fire evidence into a Bronze → Silver → Gold lakehouse.

Runs daily · about 1 hour · one F32 capacity

A fast, static-hosted web map

A React + Leaflet viewer streams immutable PMTiles releases straight from OneLake with HTTP range requests — no tile server, no database at read time.

12 toggleable layers · time-slider replay · runs in Docker

What it is NOT

- Not an authoritative fire perimeter — the fused boundary is an experimental estimate
- Not a validated spread model — the forecast is a free-burning projection
- Not Canada-wide — scope is Ontario
- Not a replacement for MNRF operations

12 map layers in four evidence groups

Satellite active fire

- VIIRS active fire (375 m, sized by FRP)
- VIIRS season replay + FRP heatmap
- Sentinel-3 SLSTR fire radiative power (1 km)

Burn extent

- Sentinel-2 RdNBR burn scars (10–20 m)
- Sentinel-1 SAR burn change (C-band, sees through smoke)
- VIIRS FRP-buffered intensity
- Fused fire boundary (ESTIMATE)

Official / validation

- Ontario in-year fire perimeters (LIO)
- MNRF validation fires
- Fire-centre labels

Forecast & danger

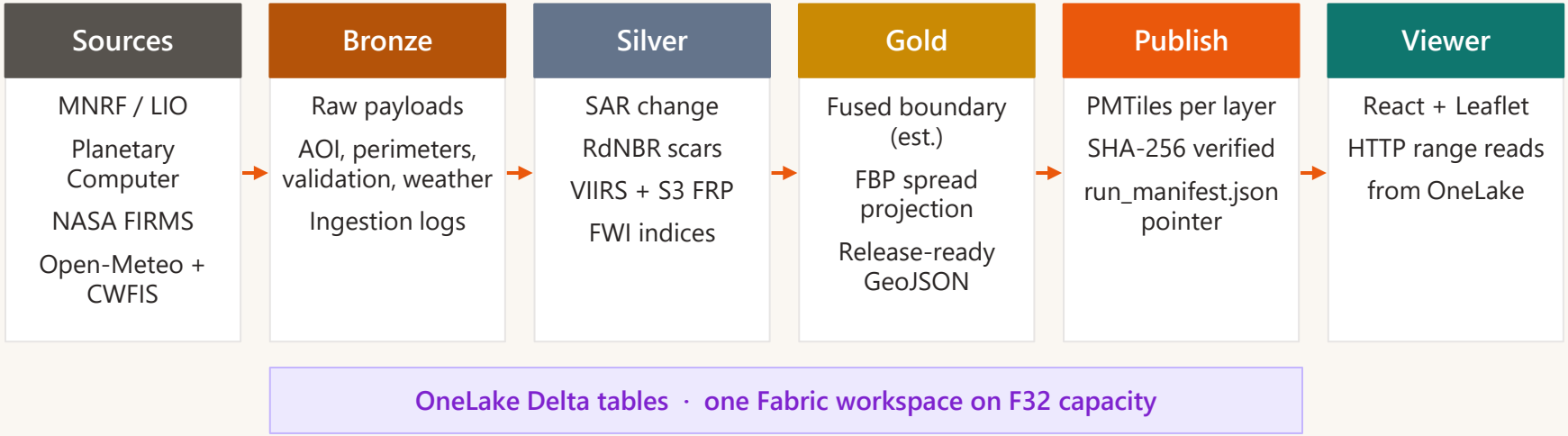
- 14-day FBP spread projection (NOT authoritative)
- Per-fire weather + Canadian FWI in the detail drawer: FFMC, DMC, DC, ISI, BUI, FWI, danger class

Complementary sensors, one lakehouse

Source	Resolution	Contribution	Trade-off
Sentinel-1 RTC SAR	10 m	All-weather burn change	Noisy; needs change detection
Sentinel-2 L2A	10–20 m	RdNBR burn-severity scars	Blocked by cloud and smoke
VIIRS (NASA FIRMS)	375 m	Active fire, ~6 passes/day	Coarse footprint
Sentinel-3 SLSTR	1 km	Fire radiative power (FRP)	Very coarse
MNRF / LIO	Vector	Active fires, perimeters, validation	Authoritative, slower
Open-Meteo + NRCan CWFIS	Point / grid	Weather + FWI indices	Local FWI fills gaps

Imagery via Microsoft Planetary Computer STAC · distances and hectares computed in EPSG:3347 (Statistics Canada Lambert)

Medallion lakehouse → immutable tiles → static viewer



Orchestration
Fabric Data Pipeline · 24 activities · gates

Compute
3 Spark environments · Runtime 1.3

Config & secrets
Variable Library · Key Vault via private endpoint

The stack, layer by layer

Data & compute

- PySpark + GeoPandas, Shapely, pyproj
- rasterio, rioxarray, xarray, dask
- pystac-client + planetary-computer
- Pure-Python tiler: mercantile + mapbox-vector-tile + pmtiles

Fabric platform

- Lakehouses (Bronze, Silver, Gold)
- 22 notebooks, 1 Data Pipeline
- Spark environments: geo-medium / large / xlarge
- Variable Library for all config

Front end

- React 18 + Vite + TypeScript
- Leaflet, protomaps-leaflet, pmtiles
- zustand state, time-slider replay
- nginx container with a OneLake proxy

Delivery & quality

- Python deployers on the Fabric REST API
- Azure DevOps pipeline + GitHub Actions
- pytest + unittest contracts, F32 offline plan
- Agent skills + issue triage for Copilot

Atomic, content-addressed publishing

1

Build Gold

Gold GeoJSON is snapshotted and hashed per layer

2

Reuse or tile

Same source + config hash? Copy the prior archive. Otherwise tile to PMTiles

3

Verify

Every archive is SHA-256 checked into a new releases/<run>/ folder

4

Swap pointer

Lock + generation check, then run_manifest.json is updated last

Why it matters Readers never see a half-written release. Rolling back means pointing the manifest at an earlier release. Unchanged layers cost nothing to republish. It's blue/green deployment for data — reusable in any static-data app.

One real run in my MCAPS subscription

61 min
end-to-end pipeline run

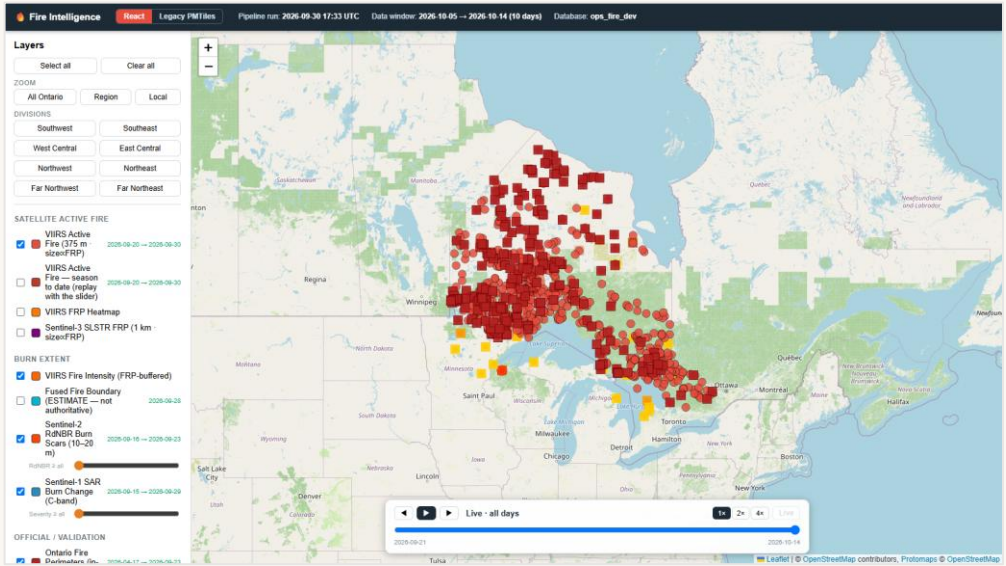
24/24
pipeline activities succeeded

11
PMTiles layers published

160
vCore peak frontier on F32

Features in this release

SAR burn-change polygons	13,237
Sentinel-3 FRP points	1,243
Sentinel-2 burn scars	977
Validation fires	801
VIIRS detections	267
Official perimeters	243



React viewer · release 2026-09-30 17:33 UTC

What people can do with it

Situational awareness

One morning map of heat, scars and official perimeters for a duty officer or briefing.

Post-fire assessment

RdNBR severity and SAR change show where and how badly land burned, even under smoke.

Perimeter QA

Overlay satellite evidence on official perimeters to spot gaps and missed spot fires.

Fire-danger context

FWI indices and weather per fire explain why a fire is behaving the way it is.

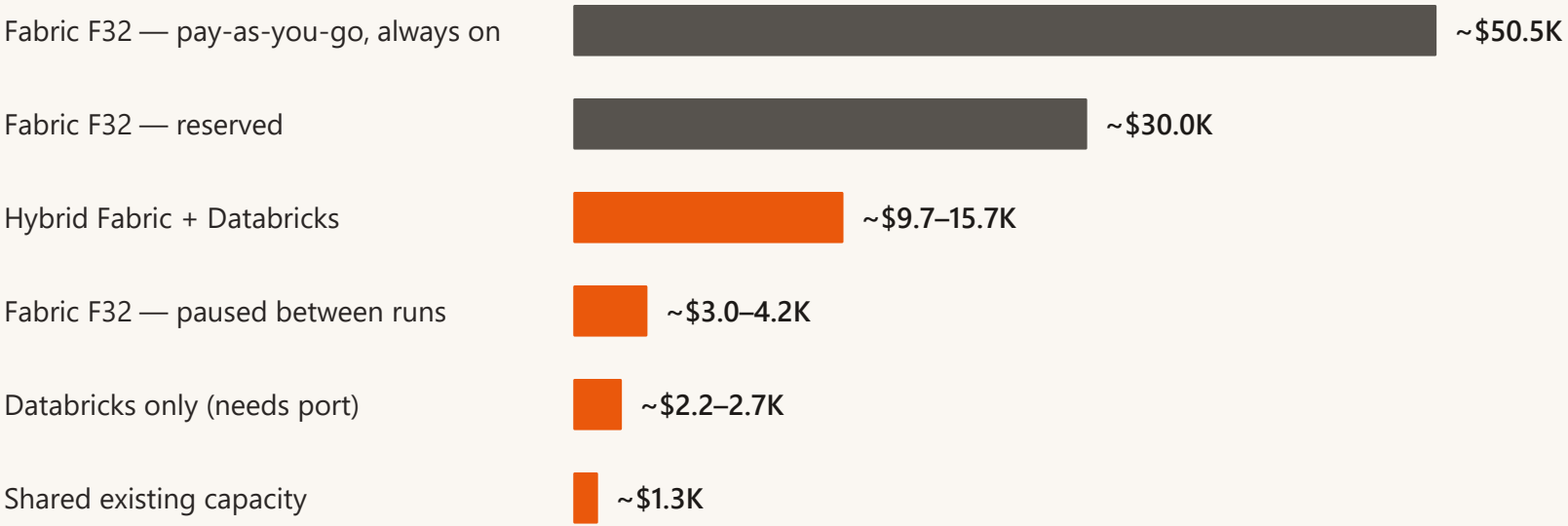
Season replay

The time slider replays the season day by day for debriefs, research and teaching.

Reference pattern

Reuse the medallion + PMTiles + manifest pattern for floods, crops, ice or any geo data.

Duty cycle, not compute, drives the bill



Estimated USD per year · measured basis ~66.7 vCore-hours per run ≈ 4.3% of an F32 day

Pause the capacity when the pipeline isn't running — the same F32 drops by >90%.

Pros and cons

Pros

- Multi-sensor evidence in one place, with provenance
- Everything in one Fabric workspace — lakehouse, Spark, orchestration
- Static PMTiles viewer: cheap, fast, CDN-friendly
- Atomic releases with instant rollback
- Config-driven: Variable Library, no hard-coded IDs
- Well tested: DAG, capacity and map-parity contracts

Cons

- Fused boundary is experimental (F1 ≈ 0.54, recall ≈ 0.42)
- Spread forecast is unvalidated
- F32 is costly if left running
- No IaC — Azure resources provisioned by hand or by agent
- Fabric REST API needed retries; Runtime 1.3 needed numpy<2
- Ontario only; FIRMS key is a prerequisite

Copilot CLI took it from repo to running

Provision	Resource group, F32 capacity, Key Vault, storage, workspace, managed private endpoints
Deploy	3 Spark environments, 22 notebooks, Variable Library value set, 24-activity pipeline
Diagnose	Read the failed Spark run, found the NumPy 2 ABI break, pinned numpy<2, republished
Adapt	Region had 0 Fabric quota → moved to Canada East; policy blocked public Key Vault → wrote secret via ARM
Verify	Pipeline 24/24 green, viewer 200, PMTiles byte-range reads confirmed

Lessons

- Give the agent the goal, not the steps
- Repo docs, tests and skills are its guardrails
- Check quotas and policy before you start
- Pin dependencies; retry idempotent calls
- You still own secrets and cost

One prompt: "Get ops fire fabric up and running end to end in my MCAPS environment."

13 · DEMO

What I'll show live (~12 min)

2 min	**Fabric workspace**	Lakehouses, notebooks, environments, Variable Library
2 min	**Pipeline run**	24 activities, dependencies, gates, durations
2 min	**Lakehouse data**	Bronze → Silver → Gold tables and the run manifest
4 min	**The map**	Layers, divisions, time-slider replay, forecast, FWI details
2 min	**Copilot CLI**	Ask it to check pipeline status or add a change, live

Questions?

Three things to take home

Fuse, don't pick

Complementary sensors beat any single source — label what is evidence and what is authoritative.

Publish like you deploy

Immutable, hashed releases plus a pointer swap make data updates atomic and reversible.

Let the agent drive

With good docs and tests, Copilot CLI can provision, deploy, debug and verify a real platform.

Source: alexaca79/ops_fire_fabric (private GitHub repo — ask me for access)